CV Intern Task: egg segmentation and tracking

Submitted by Nachiket, 1 November 2026.

Item	Link
GitHub repository (public)	https://github.com/NachiketAbhayVaidya/Egg_Detection_Pipeline
Video tracking output (Google Drive, anyone with the link)	drive.google.com/file/d/1tqbGSneG_CcvWc_d9MDTZ-C4wkDIiwCB/view

1. Summary

This report documents the full pipeline: ground truth from a pretrained SAM, a leakage-aware split, classical segmentation developed on the dev split only, a one-time evaluation, and a classical video tracker. Key results:

Part	Result
Task 1 ground truth	110 masks from pretrained SAM 2.1 Tiny with manual point prompts, checked and corrected by hand (first automatic pass was wrong for about 40% of images).
Split	87 dev / 23 eval, grouped by original image and near-duplicates (110 images, 109 groups).
Final still-image method (T12b)	Dev: IoU 0.529, Dice 0.655, precision 0.685, recall 0.786. **Eval (23 images, run once): mean IoU 0.588, median 0.666, Dice 0.712, precision 0.799, recall 0.727; 18 of 23 images above IoU 0.5, 1 below 0.1.**
Video	Classical white-egg mask + shape filters + Hungarian matching + CSRT following, 2 eggs, stable labels. Weak point: an egg held under fingers gets a tracked outline, not a true mask.

2. AI tools disclosure

An AI assistant (Claude, Anthropic) was used throughout this project to break the problem down into steps, explain the methods, and as a helping hand along the way, including drafting and debugging code. The author ran every script, inspected the outputs, made every keep/discard decision, clicked and corrected the ground-truth masks by hand, and measured the colour thresholds from real pixels.

3. Task 1: ground truth with a pretrained model

Model. SAM 2.1 Tiny (pretrained, not fine-tuned) through *SAM2ImagePredictor*.

- **First pass (automatic).** One positive click at the image centre and the highest-scoring of three masks. Reviewing the results by eye showed about 40% of masks were wrong: the click landed on the background, a hand, a shadow or a tablecloth, so the mask covered the table or hand instead of the egg. These masks were discarded as ground truth.
- **Second pass (human in the loop).** A small OpenCV click tool: left-click on every egg, right-click on anything wrongly included, one mask per request (*multimask_output=False*), keys to accept, skip, clear and invert. Photos with several eggs get a mask covering all of them.
- **Automatic checks.** A first script checked file presence, size match and egg area; a second flagged suspicious masks by shape (area above 25%, touching the border, holes, ragged outline, thin strips) so they could be re-clicked. The checks caught masks that the area test alone had passed (a mask of a table with a hole where the egg was, a thin vertical strip).

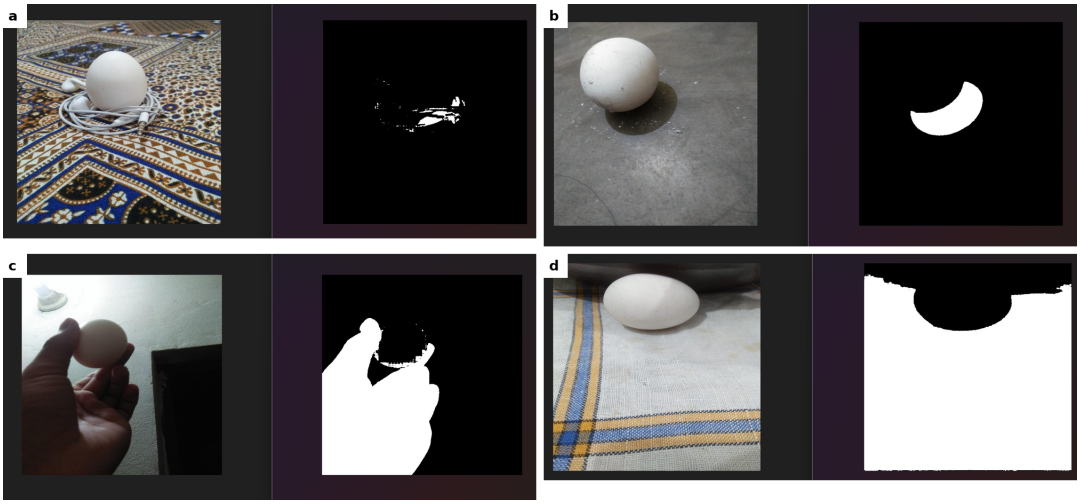


Figure 1. First automatic pass (centre click): photo on the left, SAM mask on the right. The click landed on the cloth and cables (a), the shadow (b), the hand (c) and the tablecloth (d), so the mask covers fragments, a shadow crescent, the hand and the whole background instead of the egg.

4. Leakage-aware split

Roboflow exports often contain augmented copies of one photo, and bursts of the same egg look almost identical. If copies land on both sides of a split, the exam contains pictures that were already used for tuning (leakage). The split therefore works on groups:

- Group key 1: the file name before *.rf.* (110 images gave 110 distinct originals, so no augmented copies share a name in this download).
- Group key 2: perceptual hash (*imagehash.phash*), Hamming distance of 5 or less, merged with union-find. Exactly one pair was found (*not_damaged_27* and *not_damaged_78*, distance 4; different file names, same picture, confirmed by eye). No other pair is within distance 10, so any threshold from 4 to 10 gives the same 109 groups.
- *GroupShuffleSplit(test_size=0.2, random_state=42)* on the 109 groups gave **87 dev and 23 eval images** (not exactly 88/22, because a group is never split). An assertion checks that no group is on both sides; the split is stored in *split.json*.
- Rule followed: all tuning on dev; the eval split is read by one script, run once at the end.

5. Task 2: classical segmentation and experiment log

The method uses only OpenCV, NumPy and SciPy: no neural network and no training. Each trial changes one idea and is scored on the 87 dev images against the ground truth (metrics are IoU, Dice, precision and recall).

5.1 Experiment log

Trial	What changed and why	Parameters	IoU	Dice	Prec.	Rec.	Decision
T1	Grayscale, Gaussian blur, Otsu threshold. Baseline: the simplest rule, 'egg = bright'.	blur 5x5	0.293	0.410	0.323	0.819	Kept as baseline. High recall, low precision: walls, cloth and foil are bright too. (earlier GT)
T5	K-means in LAB colour space, egg = brightest cluster. Groups pixels by colour per image instead of one global cut.	K=3, 160x160, seed 0	0.334	0.448	0.400	0.703	Kept. Less background, but the mask is scattered pieces and 'brightest' is a guess. (earlier GT)
T9	T5 mask, open/close, split into connected components, pick one by shape and position score: solidity - distance to centre - 0.3 if touching the border; area 1 to 70%; holes filled; fallback to the largest blob. Colour alone picked background.	7x7 ellipse kernel	0.377	0.473	0.497	0.614	Kept. (earlier GT)
T9	Same method, re-run after the ground-truth masks were corrected (section 3).	same	0.422	0.532	0.585	0.677	Kept. Reference for later rows.
T12	Wider candidate pool: k-means with K=3,4,5 + GrabCut seeded with a central ellipse + Canny edge ellipses. Added because the diagnosis (5.2) showed the egg was often never a candidate. Scoring unchanged.	GrabCut 5 it., ellipse seeds 20% / 40%; Canny 50/150	0.470	0.609	0.545	0.867	Kept as base. Recall up, but bloated picks hurt the good images (bucket 'good' 32 to 24).
T12b	Cascade: keep T9's own pick if its score is at least thr (confident), otherwise choose from the whole pool with an extended score = T9 score - wa*bloat + we*ellipse-fit + wg*edge strength.	thr 0.65, wa 1.5, we 1.5, wg 0.25	0.529	0.655	0.685	0.786	**Kept: final method.** Recovered the good images (24 to 36) and kept the low miss count.

* T1, T5 and the first T9 row were measured before the ground-truth correction and were not re-run; rows from the second T9 onward share the corrected ground truth.

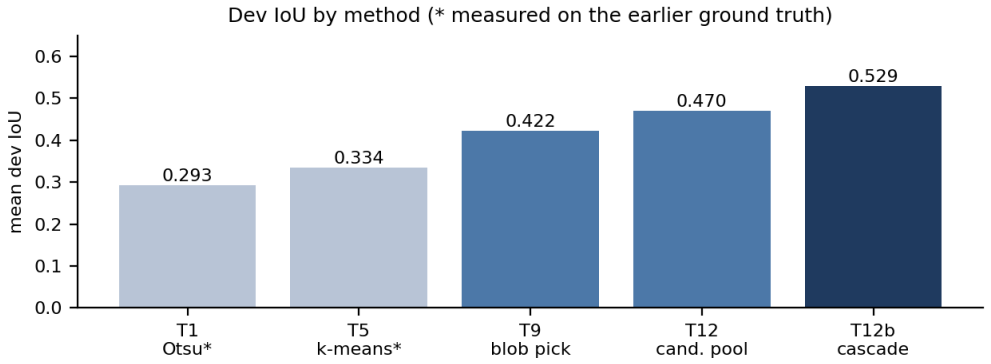


Figure 2. Mean dev IoU for each method. Light bars were measured before the ground-truth correction.

5.2 Diagnosis: where the IoU is lost

A mean IoU hides whether images are all mediocre or half perfect and half zero, and the two cases need opposite fixes. Dev images were bucketed by IoU, and for the total misses every candidate blob was scored against the ground truth to separate *generation* failures (the egg never becomes a candidate) from *scoring* failures (a good candidate exists but loses).

Step	Wrong blob (<0.1)	Bad boundary (0.1-0.6)	Good (>0.6)	Median IoU
T9, earlier ground truth	22	37	28	0.301
T9, corrected ground truth	13	42	32	0.333
T12 pool	2	61	24	0.457
T12b (final)	3	48	36	0.562

Candidate check on the total misses	Scoring failures	Generation failures	In between	Median best-candidate IoU
Earlier ground truth (22 images)	2	17	3	0.082
Corrected ground truth (13 images)	1	9	3	0.099

Result: tuning T9's scoring weights could have helped only 1 of 13 misses, so the work went into generating more candidates (T12). The pool then exposed a selection problem (T12 lost good images), which T12b fixed with the cascade. Finding that about 40% of the first ground-truth masks were wrong also changed these counts (22 to 13

total misses): many earlier 'failures' were bad answer keys, not bad predictions.

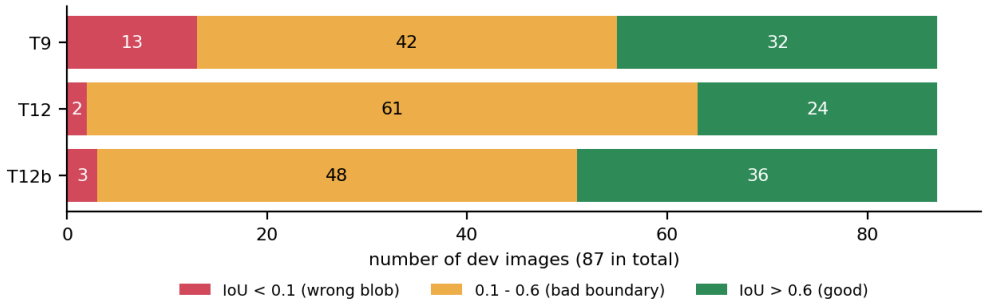


Figure 3. Dev images by IoU bucket. The candidate pool (T12) removed almost all total misses; the cascade (T12b) recovered the good images.

5.3 Tuning of T12b (dev only, one change at a time)

Setting	IoU	Dice	Prec.	Rec.	Median	IoU<0.1	IoU>0.6	Decision
default thr 0.65, wa 1.5	0.529	0.655	0.685	0.786	0.562	3	36	Kept (frozen)
thr 0.50	0.512	0.638	0.685	0.746	0.562	5	36	Worse: bad T9 picks kept
thr 0.80	0.515	0.641	0.653	0.804	0.534	3	32	Worse: good picks discarded
wa 1.0	0.529	0.655	0.685	0.786	0.562	3	36	Identical to default
wa 2.0	0.529	0.655	0.685	0.786	0.562	3	36	Identical to default

```
setting                  IoU   Dice   Prec    Rec  median  <0.1  >0.6
default thr=0.65 wa=1.5  0.529 0.655  0.685  0.786  0.562    3    36
thr=0.50                 0.512 0.638  0.685  0.746  0.562    5    36
thr=0.80                 0.515 0.641  0.653  0.804  0.534    3    32
wa=1.0                   0.529 0.655  0.685  0.786  0.562    3    36
wa=2.0                   0.529 0.655  0.685  0.786  0.562    3    36
```

Figure 4. Output of tune.py (dev only).

The rule fixed before looking: keep the default unless a setting beats it by at least 0.01 IoU. None did. *thr* is the sensitive knob (too low keeps bad T9 picks, too high discards good ones). *wa* had no effect at all at the tested values, so the bloat penalty never changes which candidate wins on this data; it could be removed or tested on images where GrabCut over-grabs.

6. Task 3: evaluation and analysis

The frozen T12b (thr 0.65, wa 1.5, we 1.5, wg 0.25) was run once on the 23 eval images, with no retuning afterwards. Predictions are the method's masks; ground truth is the corrected SAM-guided masks. An empty prediction counts as IoU 0 (none occurred).

Metric	Dev mean (87)	Eval mean (23)	Eval median
IoU	0.529	0.588	0.666
Dice	0.655	0.712	n/a
Precision	0.685	0.799	n/a
Recall	0.786	0.727	n/a
Images with IoU > 0.5	n/a	18 of 23 (78%)	
Images with IoU < 0.1	3 of 87	1 of 23	

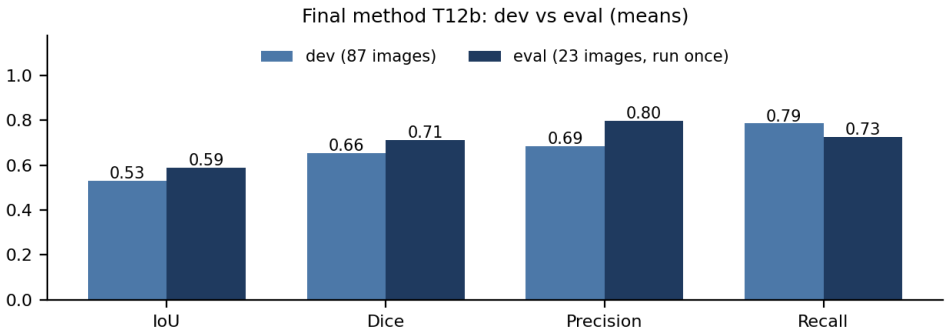


Figure 5. Final method, dev vs eval means.

```
EVAL n = 23
mean IoU 0.5877 | median 0.6658
mean Dice 0.7117
precision 0.7988 recall 0.7267
IoU > 0.5: 18 of 23 | IoU < 0.1: 1
```

Figure 6. Output of eval_final.py (23 eval images, run once).

Reading the numbers. Eval is higher than dev. With only 23 images, a few easy photos move the mean by several points, so the honest reading is that dev and eval agree within noise and nothing suggests overfitting; it is not evidence that the method generalizes better. Precision (0.80) is higher than recall (0.73): when the egg is found the mask is tight, and the misses are mostly partial masks. Median IoU (0.67) is above the mean (0.59), so a few failures pull the average down.

Worst five eval images (lowest IoU):

Image	IoU
not_damaged_104	0.00
not_damaged_117	0.17
not_damaged_128	0.27
not_damaged_133	0.34
not_damaged_142	0.48

The overlay images (prediction shaded red, true egg outlined green, plus an error map with false alarms red, missed egg blue, correct white) are produced by *make_overlays.py* and stored in outputs/eval/overlays.

6.1 Failure analysis

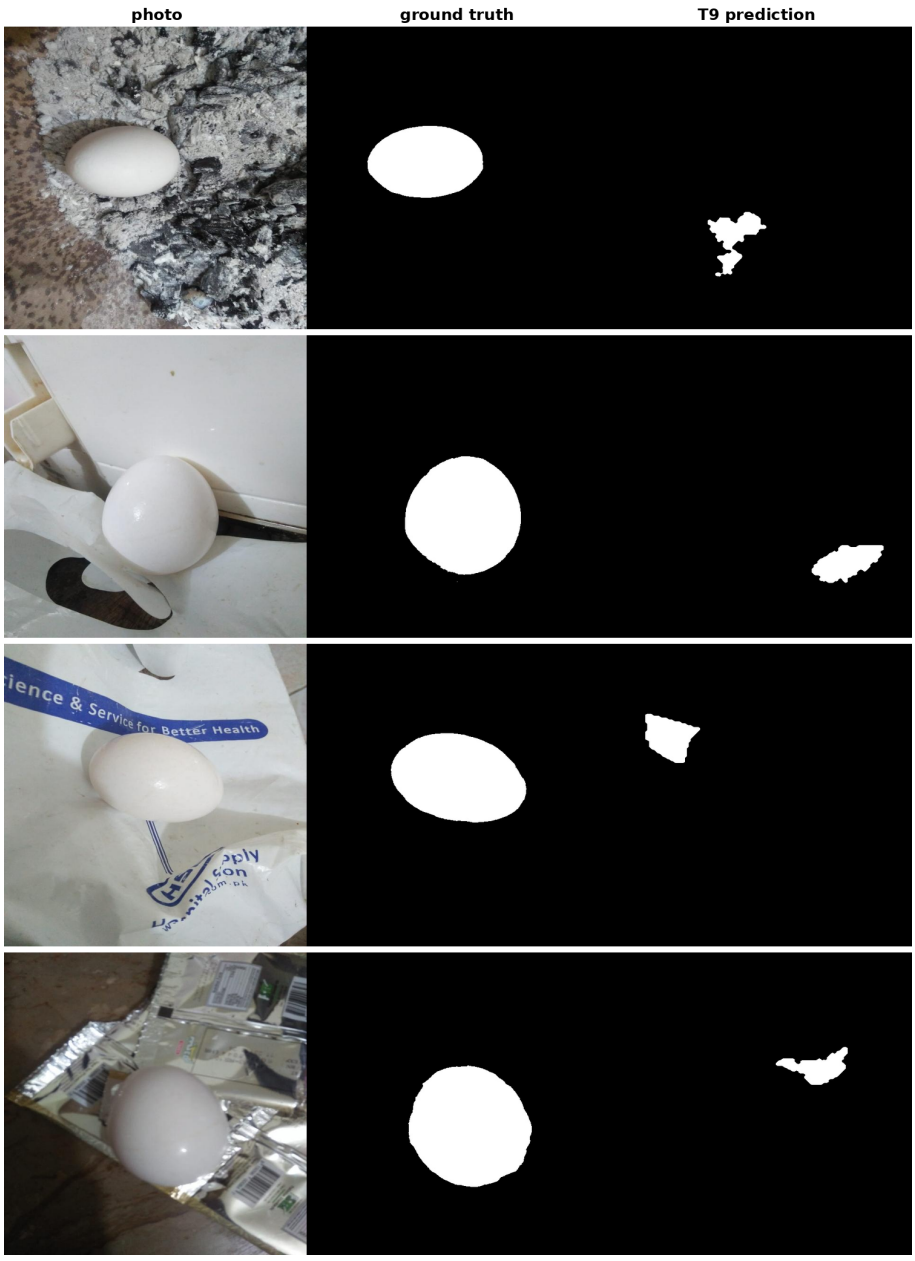


Figure 7. T9 on dev images: photo, ground truth, T9 prediction. The egg sits on ash, white plastic, a printed bag and foil; the predicted blob is a ragged fragment away from the egg because the egg and its surroundings share the same colour cluster.

- **Pale egg on a pale or busy background.** In the worst T9 cases (Figure 7) the prediction was a ragged fragment far from the egg (white plastic bags, a foil wrapper, ash, patterned cloth). K-means groups by colour, so an egg that looks like its surroundings lands in the same cluster as the background and never becomes a candidate on its own. The candidate check supports this: 9 of the 13 total misses on dev had no candidate above IoU 0.3. T12 reduced total misses from 13 to 2 by adding GrabCut and edge-ellipse candidates.
- **Right egg, wrong extent.** The largest group on dev is the bad-boundary bucket (48 of 87 for T12b). T12 showed the cause clearly: recall 0.87 with precision 0.55 means masks swallow the egg plus surroundings, because GrabCut and ellipse candidates are large and the original scoring did not penalize that. The cascade and the bloat, ellipse-fit and edge terms recovered part of this.
- **Several eggs in one photo.** The method returns one blob by design, while the ground truth contains every egg, so IoU is capped on multi-egg photos and instance-level detection scores were not computed. The video code splits touching eggs (distance transform plus watershed), but that step is not part of the still-image evaluation.
- **Ground-truth uncertainty.** The first automatic masks were wrong for about 40% of images and had to be rebuilt by hand. Remaining mistakes in the corrected masks would show up as unexplained low scores, so a

share of the failures above may belong to the answer key.

6.2 Potential improvements

- Candidate generation for pale eggs: closed-contour ellipse fitting with RANSAC or a Hough-style search, and clustering on several channels (L only, a/b only, saturation), since colour grouping is the weak link.
- Boundary refinement: GrabCut seeded from the chosen ellipse (shrunk ellipse as sure foreground, ring as probable). This is untested here.
- Multi-egg handling in stills: reuse the video's watershed split, and add instance-level precision and recall plus boundary F-score to the evaluation.
- A larger eval set and cross-validation inside dev, because 23 images cannot separate settings that differ by 0.01 IoU. A learned detector would be the next step for generality, but it is outside the classical scope of this task.

6.3 Honest notes on the evaluation

- Tuning used dev only and the eval script ran once with frozen settings; no eval prediction was seen before freezing.
- Ground-truth masks for all 110 images were corrected before the eval run, using shape checks and manual clicks; this was not guided by any eval prediction.
- The candidate-pool and cascade design (T12, T12b) was adapted from an earlier prototype of this project that had been evaluated on a different, partly overlapping split (88/22), so some eval images here were dev images there. The design was therefore not completely blind, and the eval numbers should be read with that in mind.

7. Task 4: video tracking

Output video: tracked video on Google Drive. Input: the video linked in the task, 640x360 pixels, 1440 frames at 24 fps (60 s). It starts with a black lead-in (about frames 0 to 220) and has explanatory text slides near the end with no eggs on screen. Two white eggs sit on a dark table; a hand touches, lifts and spins them. The camera is fixed.

7.1 Method

- **Why not the still-image method.** T12b assumes a centred egg, returns one blob and treats each frame alone; the video has two eggs that get lifted and moved, so a video-specific detector was built.
- **White-egg mask.** In LAB, a pixel is egg if L > 210 and chroma (sqrt((a-128)^2 + (b-128)^2)) < 10, then open and close with a 9x9 ellipse. The thresholds come from clicking real pixels (table below), not from guessing.
- **Blob selection.** Connected components; touching eggs are split with a distance transform and watershed; each part is kept only if its area is 0.5 to 25% of the frame, solidity is at least 0.85 and the fitted-ellipse aspect ratio is at most 2.0; the convex hull fills fingertip notches. At most two blobs, ranked by solidity and closeness to a 1.3 egg aspect ratio. There is no centre prior.
- **Identity.** The left egg in the first frame with two blobs is ID 1. Each frame, Hungarian matching (*linear_sum_assignment*) assigns blobs to IDs by distance to the ID's last position. A blob can only take an ID within a gate of (1.0 + 0.5 x frames missed) egg widths, which stops far-away blobs from stealing an ID. Both eggs are white, so identity comes from position only.
- **Missing detections.** When no blob matches, an OpenCV CSRT tracker (restarted on every real detection) follows the egg and the last outline is moved with it; it is capped at 120 frames and at 1.5 to 2 egg widths from the last real detection, and an outline is shown for at most 12 frames if following fails. Frames drawn this way are labelled *tracked*: they follow the egg but are not segmented.
- **Guards.** A hard scene-cut check (mean absolute frame difference above 40) stops following across cuts; an outline whose lowest point is above row 140 is rejected (the table edge is at about row 129 in the 360-pixel-high frame), which removes the pale wall.

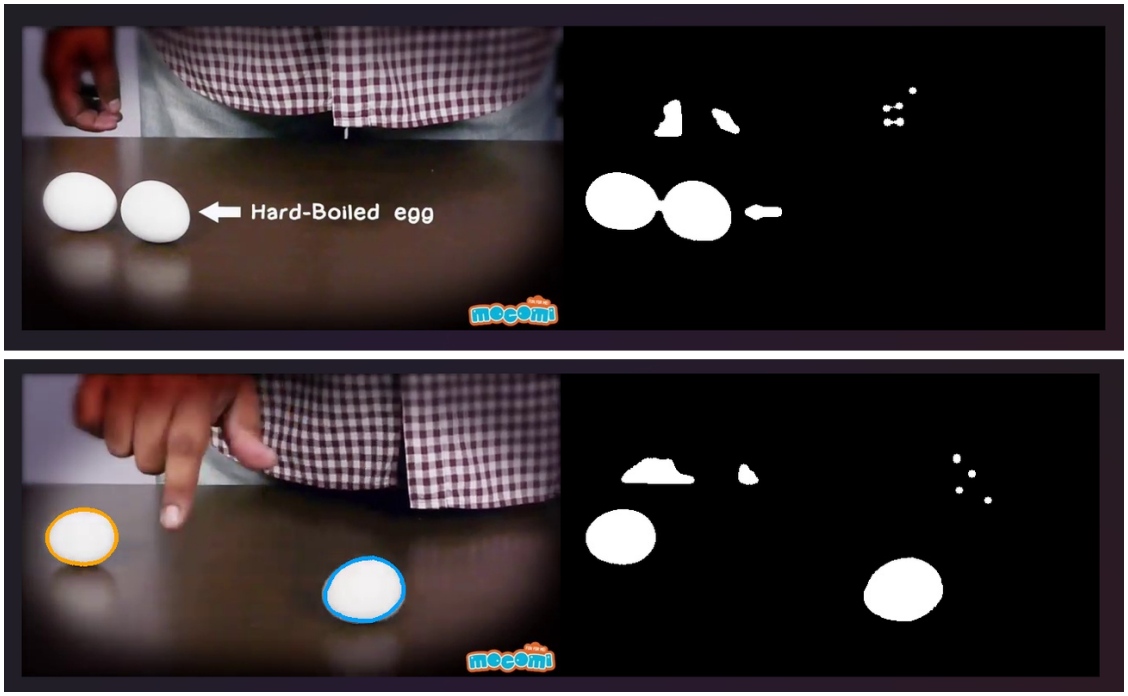


Figure 8. White-egg mask on sample frames: frame with the selected blobs outlined (left) and the raw mask (right). Top: the two touching eggs merge into one blob with the arrow, so the strict shape filter found no egg; this is what the watershed split and hull fill fixed. Bottom: separated eggs are both found, and the shirt, wall and logo fragments are rejected by size and shape.

7.2 Measured pixel values used for the thresholds

Class	L (lightness)	Chroma	Use
Egg	220 to 245	0 to 3.2	accept
Pale wall	197 to 199	5.8 to 7.1	rejected by L_MIN (first set to 180, which let the wall in; raised to 210)
Table	19 to 69	6 to 13	rejected by L_MIN
Skin	39 to 107	25 to 36	rejected by L_MIN and chroma
Fingernail	120 to 124	20 to 25	rejected by L_MIN and chroma

7.3 Attempts that failed or were reverted

Attempt	What happened	Decision
Strict shape filter only (first version)	The two touching eggs formed one merged blob with low solidity; nothing was detected on the opening shot.	Fixed with the watershed split and hull fill.
Relaxed shape limits for lost tracks (solidity 0.60, aspect 3.0, area 0.2%)	Accepted pale wall fragments as eggs and missed a fully visible egg in the same frame.	Discarded
Reject blobs whose centre is above the table edge, plus a cap on the matching gate	Eggs lifted above the edge were rejected and eggs lost for a long time were never re-found.	Discarded
Drawing the last outline frozen in place when detection fails	Outlines stayed in the air or on the wall after the egg moved or the scene changed.	Replaced by CSRT following with drift cap, scene-cut guard and a time limit.
Table-edge rule on the box centre only	Wall outlines came back at row 130; the rule is applied to the lowest point of the outline that is drawn, with a separate row for the drawn guide line.	Kept



Figure 9. Failure modes met while developing the tracker. (a) Frozen outlines left on a text slide with no eggs. (b) A followed outline drifting onto the pale wall. (c) Relaxed shape limits: an outline on a wall patch while the visible egg is missed. (d) The remaining limitation: an egg under the fingers gets a tracked outline near it, not a true mask.

7.4 Results and limitations

- Labels did not swap in any frame that was inspected: Egg 1 and Egg 2 kept their identities through the hand contact and put-down moments.
- An early version, counted over all 1440 frames, had real detections in 621 frames for Egg 1 (43.1%) and 664 for Egg 2 (46.1%). This understates performance, because about 220 black lead-in frames and the closing text slides contain no eggs. Later changes were judged by watching the rendered video.
- **Main limitation.** When fingers cover most of an egg, only a sliver is still white, so no colour rule can segment it. Those frames show a tracked outline near the egg, which can drift onto fingers; see Figure 9d.
- The method relies on a fixed camera, white eggs on a dark surface, and exactly two eggs, and is not expected to work on an arbitrary egg video without re-measuring the thresholds.
- Possible next steps: background subtraction from the fixed camera to add a motion cue, or a promptable video model such as the SAM 2 video predictor as a labelled comparison (not used for the classical result).

8. How to reproduce

The dataset is the Roboflow project linked in the task brief: download the zip, put every image (ignore its own train/valid/test folders) flat into *data/all_images*. Put the video at the path set in *VIDEO* in *src/video_track.py*. Run everything from the project root (paths are relative).

```
python -m venv myenv
myenv\Scripts\activate
pip install -r requirements.txt
git clone https://github.com/facebookresearch/sam2.git third_party/sam2
cd third_party/sam2 && pip install -e . && cd ../..
# download sam2.1_hiera_tiny.pt into checkpoints/ (link in the SAM 2 README)

python src/step1_split.py              # grouped 87/23 split -> split.json
python src/step2_sam_masks.py          # first-pass SAM masks (centre click)
python src/step2b_click_masks.py       # click tool: left = egg, right = not egg
python src/check_masks2.py             # flags suspicious masks -> flagged.txt

python src/evaluate.py t1_otsu         # also: t5_kmeans  t9_select  t12_pool  t12b
python src/diagnose.py t12b            # IoU buckets and worst dev panels
python src/diagnose_candidates.py      # generation vs scoring failures
python src/tune.py                     # thr / wa sweep, dev only

python src/eval_final.py               # RUN ONCE: 23 eval images, frozen settings
python src/eval_worst.py               # worst 5 eval panels
python src/make_overlays.py            # red/green overlays and error maps for eval

python src/video_step1.py              # frame count, fps, sample frames
python src/video_step2.py              # white-mask debug images
python src/video_render.py             # tracks and writes outputs/video/tracked_rebuild.mp4
```